TP 2 : Construire un mini RAG avec LangChain, FAISS, PromptTemplate et Groq

Module : NLP

TP guidé de l'installation jusqu'au test sur des PDF

Nom et prénom		**Groupe**	
Date		**Durée**	2h00

1. Objectifs du TP

À la fin de ce TP, l'étudiant doit être capable de :

— expliquer les deux phases d'un système RAG : indexation et question-réponse ;
— charger des documents PDF en Python ;
— découper les documents en *chunks* ;
— créer des embeddings et construire un index **FAISS** ;
— utiliser **LangChain** et **PromptTemplate** pour formater une requête ;
— interroger un modèle **Groq** depuis Python ;
— tester et analyser la qualité d'un mini chatbot RAG.

2. Idée générale du TP

Dans ce TP, on veut construire une application simple :

1. lire plusieurs fichiers PDF ;
2. extraire leur texte ;
3. les découper en morceaux ;
4. transformer ces morceaux en vecteurs ;
5. stocker ces vecteurs dans FAISS ;
6. poser une question ;
7. retrouver les extraits les plus pertinents ;
8. envoyer ces extraits à un modèle Groq avec un **PromptTemplate** ;
9. obtenir une réponse fondée sur les documents.

3. Rappel théorique très court

RAG signifie *Retrieval-Augmented Generation*.

Indexation :

— charger les documents ;
— les découper en *chunks* ;
— calculer les embeddings ;
— stocker les vecteurs.

Question-réponse :

— transformer la question en requête ;
— récupérer les *top-k* chunks les plus proches ;

— construire un prompt avec ce contexte ;
— demander au LLM de répondre.

4. Outils utilisés

Dans ce TP, on utilise :

— **Python** pour le code ;
— **LangChain** pour organiser les briques ;
— **PyPDFLoader** pour lire les PDF ;
— **RecursiveCharacterTextSplitter** pour le chunking ;
— **HuggingFaceEmbeddings** pour calculer les vecteurs ;
— **FAISS** pour l'index vectoriel ;
— **PromptTemplate** pour formater proprement la consigne ;
— **ChatGroq** pour interroger le modèle.

5. Avant de commencer

Pré-requis :

— Python 3.10 ou 3.11 ;
— un terminal ;
— une connexion Internet ;
— des PDF à tester.

Remarque importante : sur certaines machines Windows, `faiss-cpu` peut demander un environnement Python propre. Si besoin, utiliser un environnement virtuel neuf.

6. Comment obtenir une clé API Groq ?

1. ouvrir https://console.groq.com ;
2. créer un compte ou se connecter ;
3. ouvrir la zone **API Keys** ;
4. cliquer sur **Create API Key** ;
5. copier la clé générée ;
6. la stocker dans un fichier `.env`.

Contenu du fichier `.env` :

```
GROQ_API_KEY=votre_cle_api_ici
```

Bonne pratique : ne jamais écrire la clé directement dans le code Python et ne jamais la partager.

7. Organisation du projet

Créer un dossier de travail avec la structure suivante :

```
tp_rag/
|-- data/
| |-- chapitre_rag.pdf
| |-- chapitre_prompt_engineering.pdf
```

```
| |-- autres_documents.pdf
|-- .env
|-- rag_pdf_groq.py
```

Important : le dossier `data/` doit contenir les PDF à indexer et à interroger.
Conseil pour ce TP : utiliser au moins deux PDF du module, par exemple :

- `chapitre_rag.pdf`
- `chapitre_prompt_engineering.pdf`

8. Installation des dépendances

Sous Windows :

```
python -m venv .venv
.venv\Scripts\activate
pip install --upgrade pip
pip install langchain langchain-core langchain-community langchain-text-splitters
pip install langchain-huggingface langchain-groq
pip install faiss-cpu sentence-transformers pypdf python-dotenv
```

Sous Linux / macOS :

```
python3 -m venv .venv
source .venv/bin/activate
pip install --upgrade pip
pip install langchain langchain-core langchain-community langchain-text-splitters
pip install langchain-huggingface langchain-groq
pip install faiss-cpu sentence-transformers pypdf python-dotenv
```

9. Test rapide de la clé Groq

Avant de lancer le vrai TP, il faut vérifier que la clé est reconnue.

```
python -c "from dotenv import load_dotenv; import os; load_dotenv(); print(os.getenv('GROQ_API_KEY') is not None)"
```

Résultat attendu :

- `True` si la clé est bien chargée ;
- `False` si le fichier `.env` est absent ou mal rempli.

10. Code complet du TP

Enregistrer le code Python dans un fichier nommé `rag_pdf_groq.py`.

Listing 1 – Code complet du mini RAG sur PDF avec LangChain, FAISS et Groq

```
import os
from pathlib import Path

from dotenv import load_dotenv
from langchain_community.document_loaders import PyPDFLoader
from langchain_community.vectorstores import FAISS
from langchain_core.prompts import PromptTemplate
from langchain_groq import ChatGroq
from langchain_huggingface import HuggingFaceEmbeddings
from langchain_text_splitters import RecursiveCharacterTextSplitter


DATA_DIR = Path("data")
EMBEDDING_MODEL = "sentence-transformers/all-MiniLM-L6-v2"
GROQ_MODEL = "llama-3.3-70b-versatile"
CHUNK_SIZE = 800
CHUNK_OVERLAP = 150
TOP_K = 3
```

```python

def load_pdf_documents(data_dir: Path):
    pdf_paths = sorted(data_dir.glob("*.pdf"))

    if not pdf_paths:
        raise FileNotFoundError(
            "Aucun PDF trouve dans le dossier 'data'. "
            "Ajoutez par exemple chapitre_rag.pdf et chapitre_prompt_engineering.pdf."
        )

    documents = []

    for pdf_path in pdf_paths:
        loader = PyPDFLoader(str(pdf_path))
        pages = loader.load()

        for page in pages:
            page.metadata["source"] = pdf_path.name

        documents.extend(pages)

    return documents


def split_documents(documents):
    splitter = RecursiveCharacterTextSplitter(
        chunk_size=CHUNK_SIZE,
        chunk_overlap=CHUNK_OVERLAP,
        separators=["\n\n", "\n", ". ", " ", ""],
    )
    return splitter.split_documents(documents)


def create_vectorstore(chunks):
    embeddings = HuggingFaceEmbeddings(model_name=EMBEDDING_MODEL)
    return FAISS.from_documents(chunks, embeddings)


def build_prompt():
    template = """
Tu es un assistant pedagogique specialise en IA generative.
Tu dois repondre uniquement a partir du contexte fourni.

Consignes importantes :
- Reponds en francais.
- Si l'information n'est pas presente dans le contexte, dis clairement :
  "Je ne trouve pas cette information dans les documents fournis."
- Donne une reponse claire, structuree et concise.
- Termine par une ligne "Sources :" avec les fichiers utilises.

Contexte :
{context}

Question :
{question}
"""
    return PromptTemplate.from_template(template)


def format_context(docs):
    parts = []

    for i, doc in enumerate(docs, start=1):
        source = doc.metadata.get("source", "source_inconnue")
        page = doc.metadata.get("page", "?")
        if isinstance(page, int):
            page = page + 1

        parts.append(
            f"[Extrait {i} | source={source} | page={page}]\n{doc.page_content}"
        )

    return "\n\n".join(parts)

```

```python

def format_sources(docs):
    unique_sources = []

    for doc in docs:
        source = doc.metadata.get("source", "source_inconnue")
        page = doc.metadata.get("page", "?")
        if isinstance(page, int):
            page = page + 1

        item = f"{source} (page {page})"
        if item not in unique_sources:
            unique_sources.append(item)

    return ", ".join(unique_sources)


def answer_question(question, retriever, llm, prompt):
    docs = retriever.invoke(question)
    context = format_context(docs)
    sources = format_sources(docs)

    final_prompt = prompt.format(context=context, question=question)
    response = llm.invoke(final_prompt).content

    if "Sources :" not in response:
        response = response.strip() + f"\n\nSources : {sources}"

    return response, docs


def main():
    load_dotenv()

    groq_api_key = os.getenv("GROQ_API_KEY")
    if not groq_api_key:
        raise EnvironmentError(
            "La variable GROQ_API_KEY est absente. "
            "Ajoutez-la dans un fichier .env ou dans vos variables d'environnement."
        )

    print("Chargement des PDF...")
    documents = load_pdf_documents(DATA_DIR)
    print(f"Nombre total de pages chargees : {len(documents)}")

    print("Decoupage en chunks...")
    chunks = split_documents(documents)
    print(f"Nombre total de chunks : {len(chunks)}")

    print("Creation des embeddings et de l'index FAISS...")
    vectorstore = create_vectorstore(chunks)
    retriever = vectorstore.as_retriever(search_kwargs={"k": TOP_K})

    llm = ChatGroq(
        model=GROQ_MODEL,
        temperature=0,
        api_key=groq_api_key,
    )
    prompt = build_prompt()

    print("\nAssistant RAG pret.")
    print("Tapez votre question ou 'quit' pour quitter.\n")

    while True:
        question = input("Question > ").strip()

        if not question:
            print("Veuillez saisir une question.\n")
            continue

        if question.lower() in {"quit", "exit", "q"}:
            print("Fin du programme.")
            break

        answer, docs = answer_question(question, retriever, llm, prompt)
```

```
169
170         print("\n--- Reponse ---")
171         print(answer)
172
173         print("\n--- Chunks recuperes ---")
174         for i, doc in enumerate(docs, start=1):
175             source = doc.metadata.get("source", "source_inconnue")
176             page = doc.metadata.get("page", "?")
177             if isinstance(page, int):
178                 page = page + 1
179
180             preview = doc.page_content[:250].replace("\n", " ")
181             print(f"{i}. {source} | page {page}")
182             print(f" {preview}...")
183
184         print()
185
186
187 if __name__ == "__main__":
188     main()
```

11. Explication du code pas à pas

11.1. Chargement des bibliothèques

Le script importe :

- `load_dotenv` pour lire la clé Groq ;
- `PyPDFLoader` pour ouvrir les PDF ;
- `RecursiveCharacterTextSplitter` pour faire le chunking ;
- `HuggingFaceEmbeddings` pour les vecteurs ;
- `FAISS` pour la recherche vectorielle ;
- `PromptTemplate` pour construire le prompt ;
- `ChatGroq` pour générer la réponse finale.

11.2. Paramètres importants

Le script fixe dès le début :

- le dossier des PDF ;
- le modèle d'embedding ;
- le modèle Groq ;
- la taille des chunks ;
- l'overlap ;
- la valeur `TOP_K`.

Interprétation :

- `CHUNK_SIZE = 800` : chaque morceau contient environ 800 caractères ;
- `CHUNK_OVERLAP = 150` : deux morceaux se recouvrent partiellement ;
- `TOP_K = 3` : on récupère 3 extraits pertinents.

11.3. Chargement des PDF

La fonction `load_pdf_documents()` :

- cherche tous les PDF dans `data/` ;
- lit chaque page ;

— ajoute le nom du fichier dans les métadonnées ;

— fusionne toutes les pages dans une seule liste de documents.

Pourquoi les métadonnées sont utiles ? Parce qu'elles permettent d'afficher la source du passage retrouvé.

11.4. Chunking

La fonction `split_documents()` découpe les pages en morceaux plus petits.

Pourquoi ne pas garder le PDF entier ?

— un document entier est trop long ;

— la recherche est moins précise ;

— un petit extrait est plus facile à injecter dans le prompt.

11.5. Embeddings et FAISS

La fonction `create_vectorstore()` :

— transforme chaque chunk en vecteur ;

— stocke tous les vecteurs dans FAISS.

Idée essentielle : au moment d'une question, on ne compare pas directement des phrases, on compare des vecteurs.

11.6. PromptTemplate

Le `PromptTemplate` contient :

— une identité d'assistant ;

— une consigne de prudence ;

— le contexte ;

— la question.

Bonne pratique du TP : le prompt demande explicitement au modèle de dire qu'il ne sait pas si l'information n'est pas dans les documents.

11.7. ChatGroq

`ChatGroq` sert à appeler le modèle de génération.

Rôle de Groq ici :

— recevoir le contexte ;

— lire la question ;

— produire une réponse finale claire.

11.8. Boucle interactive

La fonction `main()` :

— charge la clé API ;

— charge les PDF ;

— crée les chunks ;

— crée l'index FAISS ;

— initialise le modèle Groq ;

— attend les questions de l'utilisateur.

Intérêt pédagogique : les étudiants voient clairement les deux grandes phases du RAG dans un vrai programme.

12. Exécution du programme

Lancer le script avec :

```
python rag_pdf_groq.py
```

Exemples de questions à tester :

- Qu'est-ce que le RAG ?
- Quel est le rôle de FAISS ?
- À quoi sert PromptTemplate ?
- Quelle différence entre zero-shot et few-shot ?

13. Activités pratiques à réaliser

Activité 1 : premier test

Lancer le programme et poser deux questions qui sont clairement présentes dans vos PDF.
Question 1 :

Réponse obtenue :

Question 2 :

Réponse obtenue :

Activité 2 : question absente du corpus

Poser une question qui n'est pas dans les PDF, par exemple sur un sujet totalement différent.
Question testée :

Comportement observé :

Conclusion : le prompt empêche-t-il bien le modèle d'inventer ?

Activité 3 : modifier le chunking

Changer les paramètres suivants dans le code :

- `CHUNK_SIZE = 400` puis `CHUNK_SIZE = 1000`
- `CHUNK_OVERLAP = 50` puis `CHUNK_OVERLAP = 200`

Observation : quel réglage semble le plus efficace sur vos PDF ?

Activité 4 : modifier le nombre de chunks retrouvés

Changer `TOP_K` :

- `TOP_K = 1`
- `TOP_K = 3`
- `TOP_K = 5`

Question testée :

Effet sur la qualité de la réponse :

Activité 5 : modifier le prompt

Ajouter une contrainte dans le `PromptTemplate`, par exemple :

- répondre en 5 points ;
- donner une réponse très courte ;
- citer systématiquement les sources.

Prompt modifié :

Effet observé :

Activité 6 : ajouter un nouveau PDF

Ajouter un nouveau document dans `data/`, relancer le programme puis poser une question qui dépend de ce nouveau document.

Nom du PDF ajouté :

Question testée :

Conclusion : le système a-t-il bien pris en compte le nouveau document ?

14. Questions d'analyse

1. Pourquoi faut-il découper les documents en chunks ?
2. Pourquoi utilise-t-on des embeddings ?
3. Quel est le rôle exact de FAISS ?
4. Quelle est la différence entre le retriever et le LLM ?
5. Pourquoi un `PromptTemplate` bien écrit améliore-t-il le RAG ?
6. Que se passe-t-il si la clé `GROQ_API_KEY` est absente ?
7. Quel paramètre vous semble le plus important : chunk size, overlap ou top-k ?

15. Livrable final

Le livrable final doit contenir :

— la liste des PDF utilisés ;
— une capture du dossier `data/` ;
— au moins 3 questions posées au système ;
— les réponses obtenues ;
— un test avec une question absente du corpus ;
— un test comparatif avec deux réglages différents de chunking ou de `TOP_K` ;
— une conclusion personnelle de 8 à 12 lignes.

16. Grille d'évaluation

Critère	**Barème**	**Note**
Installation correcte de l'environnement et usage de Groq	4 pts	
Compréhension de la chaîne RAG complète	4 pts	
Qualité des tests sur les PDF et des observations	4 pts	
Capacité à expliquer chunking, embeddings, FAISS et prompt	4 pts	
Qualité du livrable final et clarté de la conclusion	4 pts	
Total	**20 pts**	

17. Ressources utiles

— Groq Console : https://console.groq.com
— Groq API Docs : https://console.groq.com/docs
— LangChain : https://python.langchain.com
— FAISS : https://faiss.ai

Idée à retenir : un bon RAG ne dépend pas seulement du modèle, mais aussi de la qualité des documents, du chunking, de la recherche et du prompt.